Copula (Glasgow)

Guanyu

2026-02-13

Raw data

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.1     v stringr   1.5.1
## v ggplot2   4.0.0     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(sf)
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```r
simd <- readr::read_csv("simd2020_withgeog/simd2020_withinds.csv")
```

```
## Rows: 6976 Columns: 49
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (16): Data_Zone, Intermediate_Zone, Council_area, income_rate, employmen...
## dbl (33): Total_population, Working_Age_population, SIMD2020v2_Rank, SIMD_20...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
simd_shp <- sf::st_read("simd2020_withgeog/sc_dz_11.shp")
```

```
## Reading layer `sc_dz_11' from data source
##   `/Users/chenguanyu/Desktop/SIMD20/SIMD/simd2020_withgeog/sc_dz_11.shp'
##   using driver `ESRI Shapefile'
## Simple feature collection with 6976 features and 9 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -8.650007 ymin: 54.63324 xmax: -0.7246094 ymax: 60.86077
## Geodetic CRS:  WGS 84
```

```r
simd_joined <- simd_shp %>%
  left_join(simd, by = c("DataZone" = "Data_Zone"))
joint_board_lookup <- tribble(
```

```r
  ~Council_area,                          ~Joint_Board_area,
  "East Ayrshire",                       "Ayrshire",
  "North Ayrshire",                      "Ayrshire",
  "South Ayrshire",                      "Ayrshire",
  "Scottish Borders",                    "Borders",
  "Clackmannanshire",                    "Central Scotland",
  "Falkirk",                             "Central Scotland",
  "Stirling",                            "Central Scotland",
  "Dumfries and Galloway",               "Dumfries and Galloway",
  "Argyll and Bute",                     "Dunbartonshire and Argyll & Bute",
  "East Dunbartonshire",                 "Dunbartonshire and Argyll & Bute",
  "West Dunbartonshire",                 "Dunbartonshire and Argyll & Bute",
  "Fife",                                "Fife",
  "Aberdeen City",                       "Grampian",
  "Aberdeenshire",                       "Grampian",
  "Moray",                               "Grampian",
  "Glasgow City",                        "Glasgow",
  "Highland",                            "Highlands and Islands",
  "Na h-Eileanan an Iar",                "Highlands and Islands",
  "North Lanarkshire",                   "Lanarkshire",
  "South Lanarkshire",                   "Lanarkshire",
  "East Lothian",                        "Lothian",
  "City of Edinburgh",                   "Lothian",
  "Midlothian",                          "Lothian",
  "West Lothian",                        "Lothian",
  "Orkney Islands",                      "Orkney and Shetland",
  "Shetland Islands",                    "Orkney and Shetland",
  "East Renfrewshire",                   "Renfrewshire",
  "Inverclyde",                          "Renfrewshire",
  "Renfrewshire",                        "Renfrewshire",
  "Angus",                               "Tayside",
  "Dundee City",                         "Tayside",
  "Perth and Kinross",                   "Tayside"
)

simd_joined <- simd_joined %>%
  left_join(joint_board_lookup, by = c("Council_area" = "Council_area"))

# Pull out Glasgow data

indicator_cols <- c(
"income_rate", "income_count",
"employment_rate", "employment_count",
"CIF", "ALCOHOL", "DRUG", "SMR", "DEPRESS",
"LBWT", "EMERG", "Attendance", "Attainment",
"no_qualifications", "not_participating", "University",
"crime_count", "crime_rate",
"overcrowded_count", "nocentralheating_count",
"overcrowded_rate", "nocentralheating_rate",
"drive_petrol", "drive_GP", "drive_post",
"drive_primary", "drive_retail", "drive_secondary",
"PT_GP", "PT_post", "PT_retail", "broadband"
)
```

```r
simd_glasgow_raw <- simd_joined %>%
  filter(Joint_Board_area == "Glasgow") %>%
  st_drop_geometry() %>%
  select(DataZone, any_of(indicator_cols)) %>%
  mutate(
    across(
      where(is.character) & !matches("^DataZone$"),
      ~ as.numeric(str_remove(.x, "%"))
      )
    )
```

```
## Warning: There were 4 warnings in `mutate()`.
## The first warning was:
## i In argument: `across(...)`.
## Caused by warning:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
```

```r
na_summary_gla <- simd_glasgow_raw %>%
  summarise(
    across(
      all_of(indicator_cols),
      list(
        Missing_Count   = ~ sum(is.na(.)),
        Percent_Missing = ~ mean(is.na(.)) * 100
        ),
      .names = "{.col}.{.fn}")
    ) %>%
  pivot_longer(
    everything(),
    names_to  = c("Indicator", ".value"),
    names_sep = "\\.") %>%
  arrange(desc(Missing_Count))


glasgow_ind <- na_summary_gla %>%
  filter(Percent_Missing <= 10) %>%
  pull(Indicator)

simd_glasgow <- simd_glasgow_raw %>%
  select(DataZone, all_of(glasgow_ind)) %>%
  drop_na(all_of(glasgow_ind))

# Standardize for PCA & clustering

glasgow_data <- simd_glasgow %>%
  select(-DataZone) %>%
  scale() %>%
  as.data.frame()

# Domain Mapping

indicator_domains <- tribble(
  ~Indicator,                ~Domain,
```

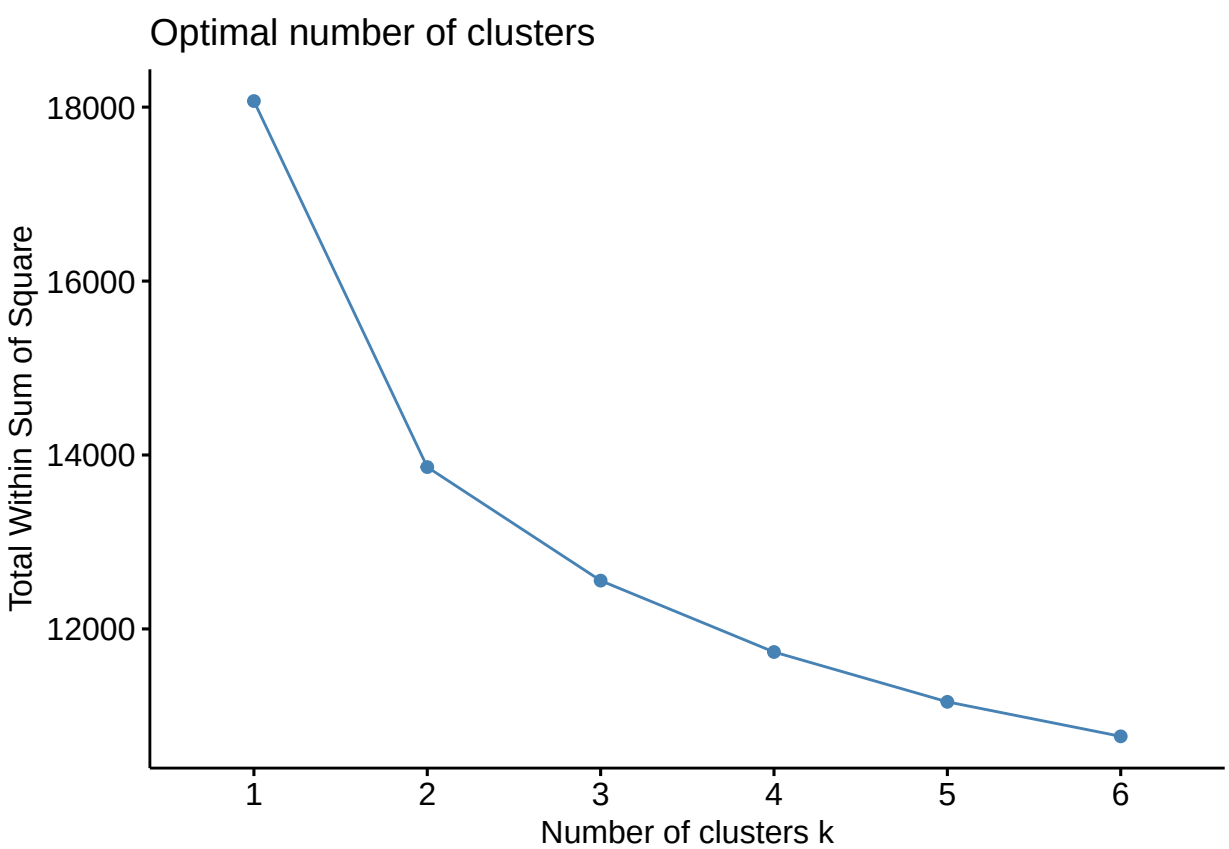
```r
  # Income
  "income_rate",              "Income",
  "income_count",             "Income",

  # Employment
  "employment_rate",          "Employment",
  "employment_count",         "Employment",

  # Health
  "CIF",                      "Health",
  "ALCOHOL",                  "Health",
  "DRUG",                     "Health",
  "SMR",                      "Health",
  "DEPRESS",                  "Health",
  "LBWT",                     "Health",
  "EMERG",                    "Health",

  # Education
  "Attendance",               "Education",
  "Attainment",               "Education",
  "no_qualifications",        "Education",
  "not_participating",        "Education",
  "University",               "Education",

  # Access
  "drive_petrol",             "Access",
  "drive_GP",                 "Access",
  "drive_post",               "Access",
  "drive_primary",            "Access",
  "drive_retail",             "Access",
  "drive_secondary",          "Access",
  "PT_GP",                    "Access",
  "PT_post",                  "Access",
  "PT_retail",                "Access",
  "broadband",                "Access",

  # Crime
  "crime_count",              "Crime",
  "crime_rate",               "Crime",

  # Housing
  "overcrowded_count",        "Housing",
  "nocentralheating_count",   "Housing",
  "overcrowded_rate",         "Housing",
  "nocentralheating_rate",    "Housing"
)

domain_vars <- indicator_domains %>%
  filter(Indicator %in% colnames(glasgow_data)) %>%
  group_by(Domain) %>%
  summarise(vars = list(Indicator)) %>%
  deframe()
```

```r
raw_ind <- indicator_domains %>%
  filter(Domain %in% c("Health", "Education", "Access")) %>%
  filter(!Indicator %in% c("Attendance")) %>%
  pull(Indicator)

rate_ind <- c("income_rate", "employment_rate", "crime_rate",
         "nocentralheating_rate", "overcrowded_rate")

raw_data <- glasgow_data %>% dplyr::select(any_of(c(raw_ind, rate_ind)))
```

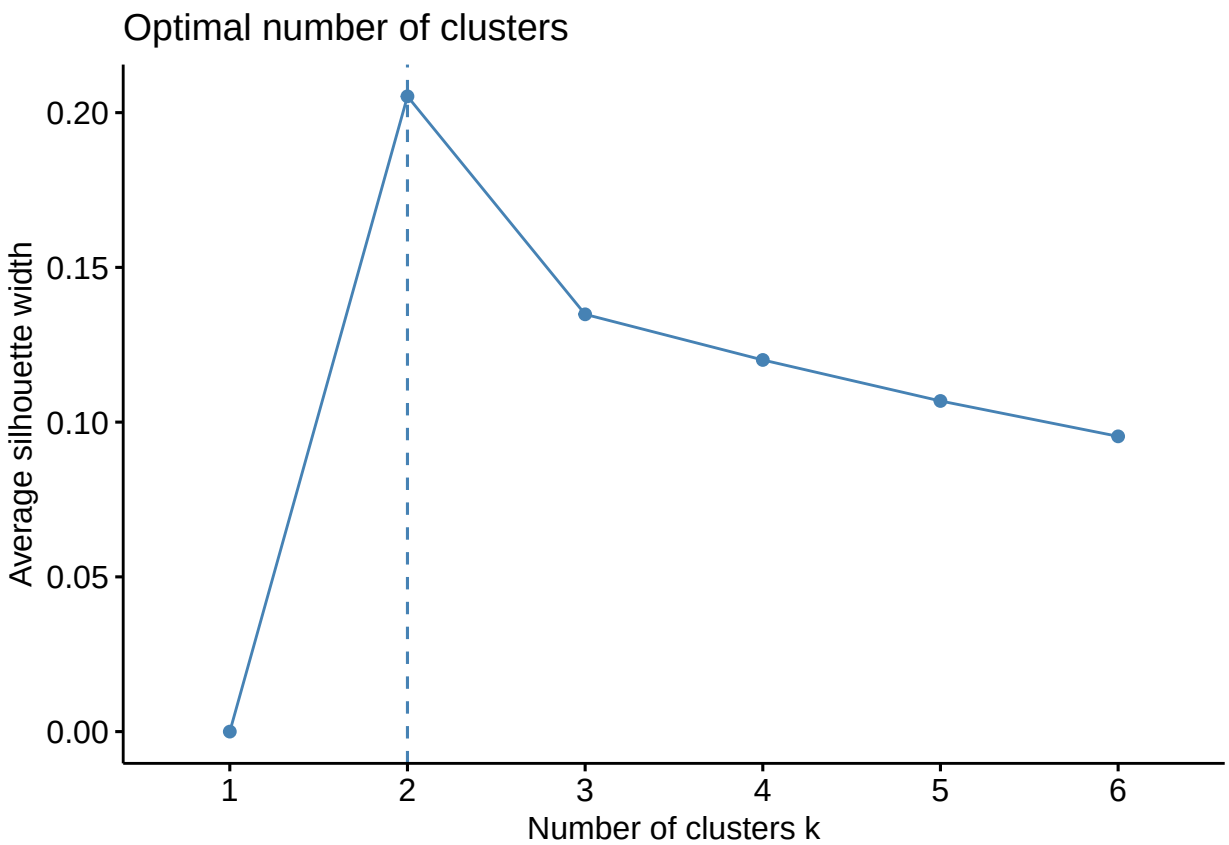
```r
set.seed(123)
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```r
# Elbow (WSS)
fviz_nbclust(raw_data, kmeans, method = "wss", k.max = 6, nstart = 25)
```




```r
# Silhouette (often more informative than elbow)
fviz_nbclust(raw_data, kmeans, method = "silhouette", k.max = 6, nstart = 25)
```




```
k <- 2
k_raw <- kmeans(raw_data, centers = k, nstart = 25)
glasgow_data <- raw_data %>%
  mutate(
    cluster_k = k_raw$cluster
  )
```

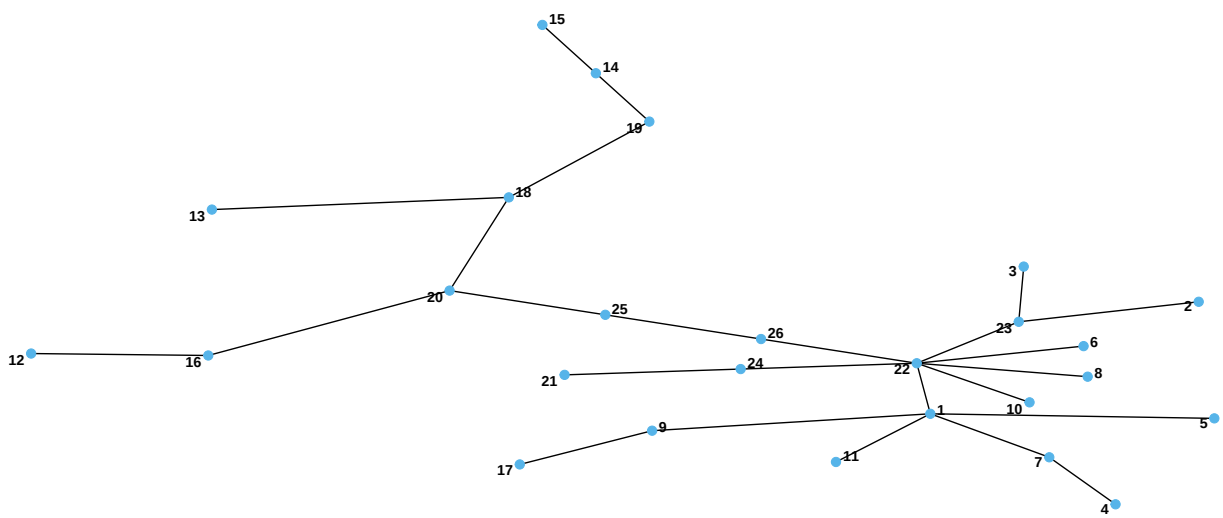
R-vine

```
## 26-dimensional vine copula fit ('vinecop')
## nobs = 696   logLik = 8176.72   npars = 122   AIC = -16109.44   BIC = -15554.91
```

```
## # A data.frame: 325 x 11
##  tree edge conditioned conditioning var_types   family rotation
##     1    1      13, 18                    c,c gaussian        0
##     1    2      15, 14                    c,c    frank        0
##     1    3       17, 9                    c,c   gumbel      180
##     1    4      14, 19                    c,c   gumbel      180
##     1    5       3, 23                    c,c    frank        0
##     1    6      19, 18                    c,c     tawn        0
##     1    7       2, 23                    c,c      bb8      180
##     1    8      12, 16                    c,c  clayton      180
##     1    9      23, 22                    c,c    frank        0
##     1   10      16, 20                    c,c      bb1      180
##        parameters df  tau loglik
##              0.89  1 0.70    554
##               1.8  1 0.19     30
##               1.2  1 0.17     33
```

```
##                2.6  1 0.62    419
##                8.4  1 0.62    362
##  1.00, 0.63, 1.99  3 0.36    140
##        8.00, 0.67  2 0.60    348
##                0.3  1 0.13     19
##                 25  1 0.85    943
##        0.27, 2.38  2 0.63    437
## # ... with 315 more rows
##
## ---
## 1 <-> CIF,   2 <-> ALCOHOL,   3 <-> DRUG,   4 <-> SMR,   5 <-> DEPRESS,   6 <-> LBWT,
## 7 <-> EMERG,   8 <-> Attainment,   9 <-> no_qualifications,   10 <-> not_participating,
## 11 <-> University,   12 <-> drive_petrol,   13 <-> drive_GP,   14 <-> drive_post,   15 <-> drive_pri
## 16 <-> drive_retail,   17 <-> drive_secondary,   18 <-> PT_GP,   19 <-> PT_post,   20 <-> PT_retail,
## 21 <-> broadband,   22 <-> income_rate,   23 <-> employment_rate,   24 <-> crime_rate,
## 25 <-> nocentralheating_rate,   26 <-> overcrowded_rate
```

Tree 1




```
## [1] 8176.722
## attr(,"df")
## [1] 122
```

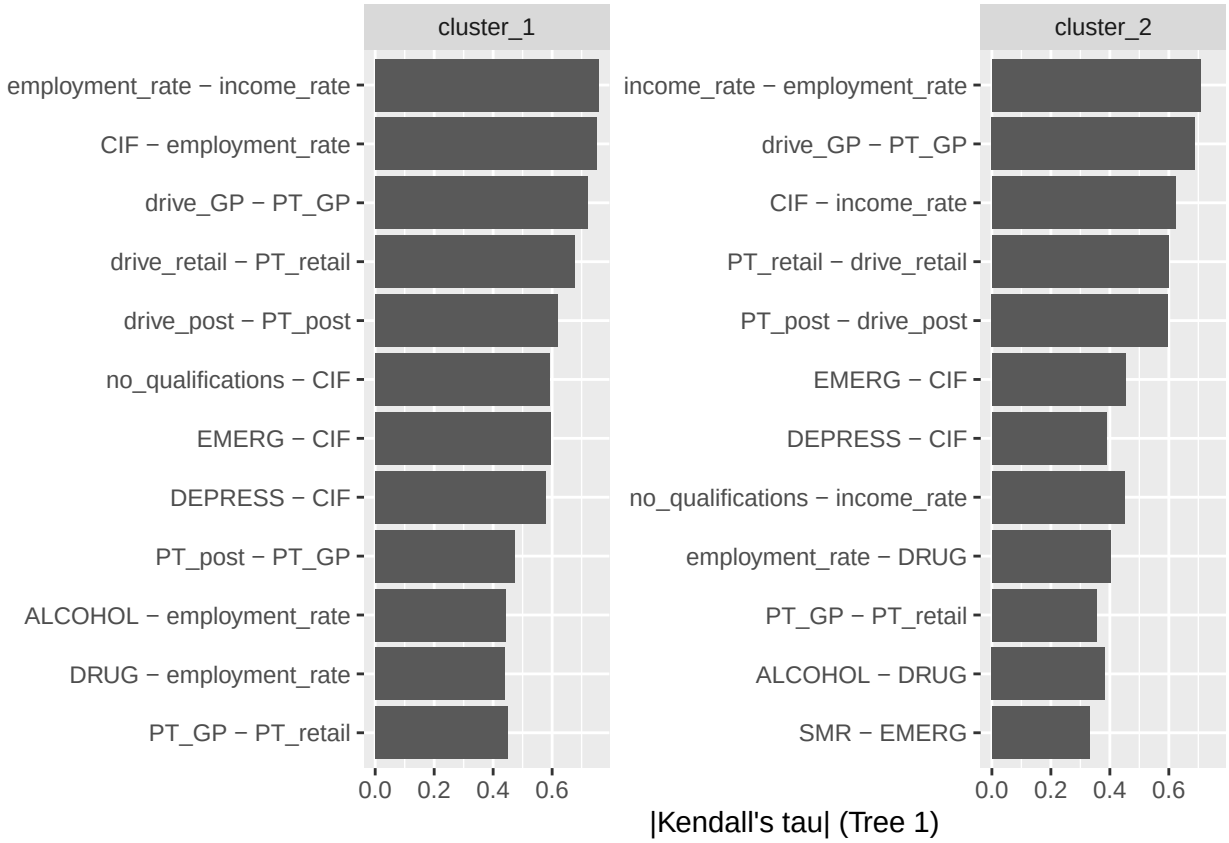
```
## [1] -16109.44
```

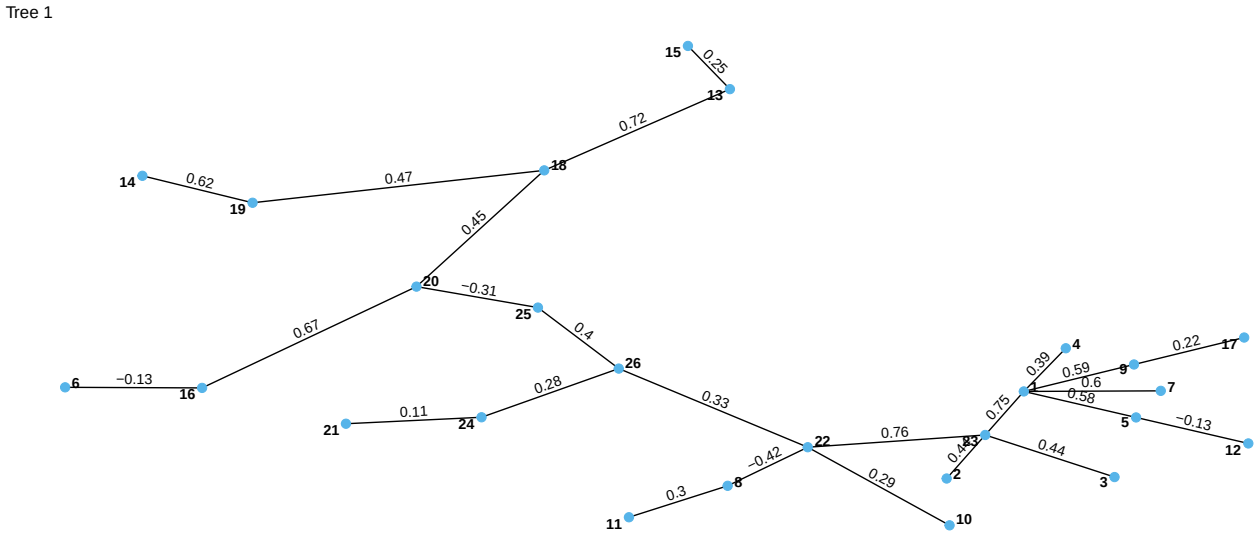
```
## [1] -15554.91
```

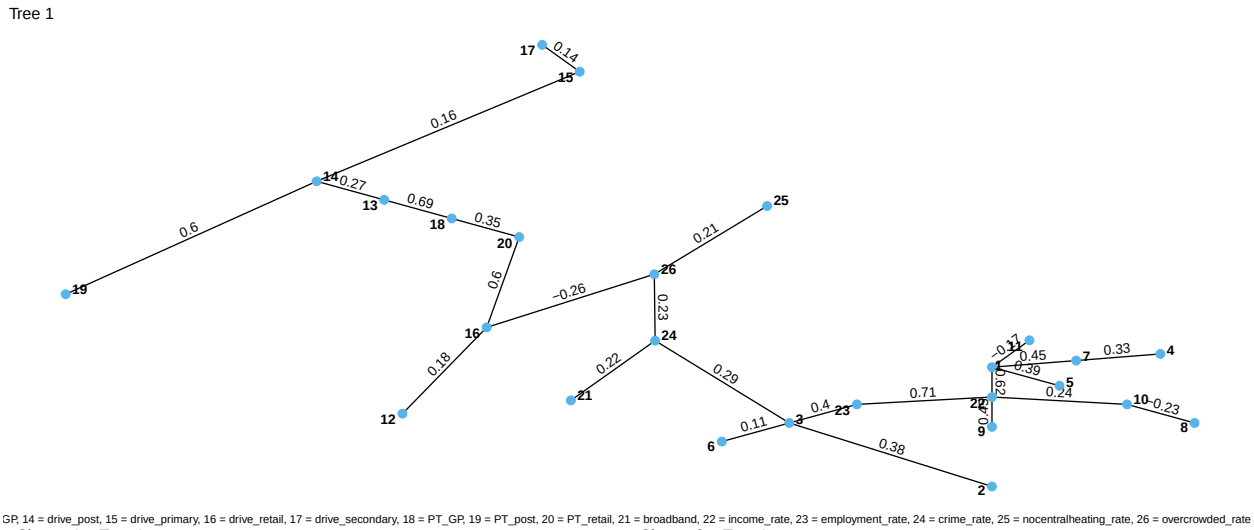
```
fit_cluster_vines <- function(df, cluster_col = "cluster_k", cores = 4) {
  clusters <- sort(unique(df[[cluster_col]]))

  res <- lapply(clusters, function(k) {
    Xk <- df %>%
      filter(.data[[cluster_col]] == k) %>%
      select(where(is.numeric)) %>%
      select(-any_of(cluster_col)) %>%
      drop_na() %>%
      as.data.frame()

    Uk <- pseudo_obs(as.matrix(Xk))
```

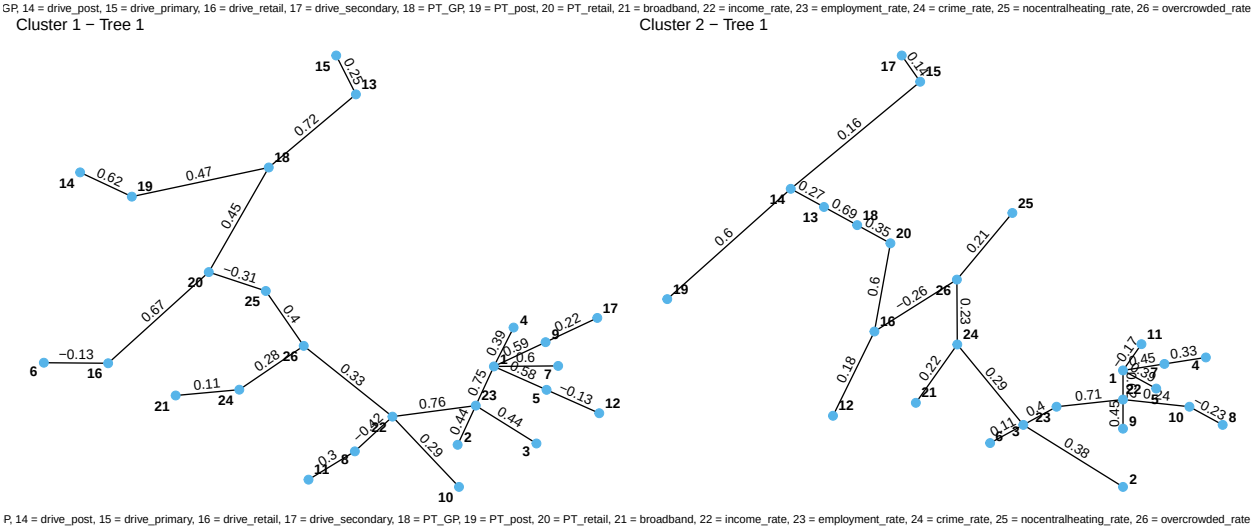
```r
    fitk <- vinecop(
      data = Uk,
      family_set = "parametric",
      selcrit = "bic",
      tree_crit = "tau",
      trunc_lvl = Inf,
      cores = cores
    )

    list(k = k, X = Xk, U = Uk, fit = fitk)
  })

  names(res) <- paste0("cluster_", clusters)
  res
}

fits <- fit_cluster_vines(glasgow_data, cores = 4)

edge_table <- function(fit) {
  sm <- summary(fit)  # data-frame-like

  vn <- attr(sm, "var_names")
  if (is.null(vn) || length(vn) == 0) vn <- paste0("V", seq_len(ncol(fit$data)))

  sm$var1 <- vn[sapply(sm$conditioned, `[[`, 1)]
  sm$var2 <- vn[sapply(sm$conditioned, `[[`, 2)]

  sm$cond_set <- sapply(sm$conditioning, function(idx) {
    idx <- idx[idx != 0]
    if (length(idx) == 0) "" else paste(vn[idx], collapse = ", ")
  })

  sm
}

edges_by_cluster <- lapply(fits, function(obj) edge_table(obj$fit))

edges_all <- bind_rows(lapply(names(edges_by_cluster), function(nm) {
  df <- edges_by_cluster[[nm]]
  df %>%
    mutate(
      cluster = nm,
      pair = paste(var1, var2, sep = " - "),
      abs_tau = abs(tau)
    )
}))

top_tree1 <- edges_all %>% filter(tree == 1) %>%
  group_by(cluster) %>% slice_max(abs_tau, n = 12, with_ties = FALSE) %>%
  ungroup()

ggplot(top_tree1, aes(x = abs_tau, y = reorder(pair, abs_tau))) +
  geom_col() +
```

```r
facet_wrap(~ cluster, scales = "free_y") +
labs(x = "|Kendall's tau| (Tree 1)", y = NULL)
```










```r
# ============================================================
# Chunk: VineCopula cluster-wise RVine + CVine + readable plots
# ============================================================

library(tidyverse)
library(VineCopula)
library(stringr)

# ----------------------------
# 0) Helpers
# ----------------------------

make_U <- function(X) {
  U <- VineCopula::pobs(as.matrix(X))
  colnames(U) <- colnames(X)
  U
}

get_cluster_X <- function(df, k, cluster_col = "cluster_k") {
  df %>%
    dplyr::filter(.data[[cluster_col]] == k) %>%
```

```r
    dplyr::select(where(is.numeric)) %>%
    dplyr::select(-any_of(cluster_col)) %>%
    tidyr::drop_na() %>%
    as.data.frame()
}

fit_one_cluster_vinecopula <- function(df, k,
                                       cluster_col = "cluster_k",
                                       type = c("RVine", "CVine"),
                                       selectioncrit = "BIC",
                                       treecrit = "tau",
                                       familyset = NA,
                                       trunclevel = NA,
                                       indeptest = TRUE,
                                       level = 0.05,
                                       cores = 4,
                                       progress = FALSE) {
  type <- match.arg(type)

  Xk <- get_cluster_X(df, k, cluster_col = cluster_col)
  Uk <- make_U(Xk)

  RVM <- VineCopula::RVineStructureSelect(
    data = Uk,
    familyset = familyset,
    type = type,                  # "RVine" or "CVine"
    selectioncrit = selectioncrit,
    indeptest = indeptest,
    level = level,
    trunclevel = trunclevel,
    treecrit = treecrit,
    cores = cores,
    progress = progress
  )

  # IMPORTANT: store variable names so plots + downstream extraction are readable
  RVM$names <- colnames(Uk)

  list(cluster = k, X = Xk, U = Uk, RVM = RVM)
}

fit_clusterwise_vines <- function(df,
                                  cluster_col = "cluster_k",
                                  clusters = NULL,
                                  type = c("RVine", "CVine"),
                                  selectioncrit = "BIC",
                                  indeptest = TRUE,
                                  level = 0.05,
                                  trunclevel = NA,
                                  familyset = NA,
                                  cores = 4) {
  type <- match.arg(type)
  if (is.null(clusters)) clusters <- sort(unique(df[[cluster_col]]))
```

```r
  fits <- purrr::map(clusters, ~fit_one_cluster_vinecopula(
    df = df, k = .x, cluster_col = cluster_col,
    type = type,
    selectioncrit = selectioncrit,
    indeptest = indeptest,
    level = level,
    trunclevel = trunclevel,
    familyset = familyset,
    cores = cores
  ))

  names(fits) <- paste0("cluster_", clusters)
  fits
}

# create fits_rvine / fits_cvine (VineCopula)
fits_rvine <- fit_clusterwise_vines(
  df = glasgow_data,
  cluster_col = "cluster_k",
  type = "RVine",
  selectioncrit = "BIC",
  indeptest = TRUE,
  level = 0.05,
  cores = 4
)

fits_cvine <- fit_clusterwise_vines(
  df = glasgow_data,
  cluster_col = "cluster_k",
  type = "CVine",
  selectioncrit = "BIC",
  indeptest = TRUE,
  level = 0.05,
  cores = 4
)


# ==========================================
# WHOLE COPY-PASTE READY VISUAL CHUNK
# - Tree plots side-by-side for cluster_1 vs cluster_2
# - Tree >= 2 shows conditioning variables as NO (node numbers), e.g. "| 1,23"
# - Tree 1 shows tau labels by default
# - tau = 0 edges are still drawn (light dashed)
# - A natural "NO → Variable" key table is shown under the plots
# ==========================================

suppressPackageStartupMessages({
  library(dplyr)
  library(tibble)
  library(stringr)
  library(igraph)
  library(ggraph)
  library(ggplot2)
```

```r
  library(patchwork)
  library(gridExtra)
  library(grid)
})
```

```
## Warning: package 'igraph' was built under R version 4.4.3
```

```r
# ---------- helper: canonical parse + formatting ----------
format_cond_no <- function(cond_idx) {
  idx <- cond_idx[!is.na(cond_idx)]
  if (length(idx) == 0) return("")
  idx <- sort(unique(idx))
  paste0("| ", paste(idx, collapse = ","))
}

# ---------- Extract edges (incl. conditional set) from print(RVM) ----------
extract_tree_edges <- function(RVM, tree = 1, var_names = RVM$names) {

  out <- capture.output(print(RVM))

  i1 <- grep(paste0("^Tree\\s+", tree, "\\b"), out)[1]
  i2 <- grep(paste0("^Tree\\s+", tree + 1, "\\b"), out)[1]
  if (is.na(i1)) stop(paste0("Could not find 'Tree ", tree, "' in print(RVM)."))
  if (is.na(i2)) i2 <- length(out) + 1

  lines <- out[(i1 + 1):(i2 - 1)]
  lines <- trimws(gsub("^#+\\s*", "", lines))
  lines <- lines[nzchar(lines)]
  lines <- lines[grepl("^[0-9]+\\s*,\\s*[0-9]+", lines)]

  if (length(lines) == 0) {
    return(tibble(
      idx1 = integer(), idx2 = integer(),
      var1 = character(), var2 = character(),
      cond_idx = list(),
      family = character(), tau = numeric(), abs_tau = numeric()
    ))
  }

  toks <- strsplit(lines, "\\s+")
  pair_tok <- vapply(toks, function(z) if (length(z) >= 1) z[1] else NA_character_, character(1))
  fam_raw  <- vapply(toks, function(z) if (length(z) >= 2) z[2] else NA_character_, character(1))
  fam_raw[is.na(fam_raw)] <- "Indep"

  tau <- suppressWarnings(as.numeric(str_match(lines, "tau\\s*=\\s*([-0-9.]+)")[, 2]))
  tau[is.na(tau)] <- 0

  # split "i,j;D" or "i,j|D"
  pair_part <- sub("[;|].*$", "", pair_tok)
  cond_part <- ifelse(grepl("[;|]", pair_tok), sub("^[^;|]+[;|]", "", pair_tok), "")

  # endpoints
  parts <- strsplit(pair_part, ",", fixed = TRUE)
  v1 <- trimws(vapply(parts, function(z) if (length(z) >= 1) z[1] else NA_character_, character(1)))
```

```r
  v2 <- trimws(vapply(parts, function(z) if (length(z) >= 2) z[2] else NA_character_, character(1)))

  idx1 <- suppressWarnings(as.integer(v1))
  idx2 <- suppressWarnings(as.integer(v2))
  if (anyNA(idx1)) idx1[is.na(idx1)] <- match(v1[is.na(idx1)], var_names)
  if (anyNA(idx2)) idx2[is.na(idx2)] <- match(v2[is.na(idx2)], var_names)

  # conditioning set indices (list column)
  cond_idx <- lapply(cond_part, function(s) {
    s <- trimws(s)
    if (!nzchar(s)) return(integer(0))
    z <- trimws(unlist(strsplit(s, ",", fixed = TRUE)))
    z <- z[nzchar(z)]
    # usually numeric; fallback to name match if not
    zi <- suppressWarnings(as.integer(z))
    if (anyNA(zi)) zi[is.na(zi)] <- match(z[is.na(zi)], var_names)
    zi[!is.na(zi)]
  })

  tibble(
    idx1 = idx1,
    idx2 = idx2,
    var1 = var_names[idx1],
    var2 = var_names[idx2],
    cond_idx = cond_idx,
    family = fam_raw,
    tau = tau,
    abs_tau = abs(tau)
  ) %>% filter(!is.na(idx1), !is.na(idx2))
}

# ---------- Shared layout across cluster_1 & cluster_2 ----------
make_reference_layout <- function(RVM1, RVM2, tree = 1, seed = 1) {
  vn <- RVM1$names
  p  <- length(vn)
  nodes <- tibble(name = as.character(seq_len(p)))

  ed1 <- extract_tree_edges(RVM1, tree = tree, var_names = vn) %>%
    transmute(from = as.character(idx1), to = as.character(idx2))
  ed2 <- extract_tree_edges(RVM2, tree = tree, var_names = vn) %>%
    transmute(from = as.character(idx1), to = as.character(idx2))

  ed_u <- bind_rows(ed1, ed2) %>% distinct()
  g_ref <- igraph::graph_from_data_frame(ed_u, directed = FALSE, vertices = nodes)

  set.seed(seed)
  xy <- igraph::layout_with_fr(g_ref)
  tibble(name = igraph::V(g_ref)$name, x = xy[, 1], y = xy[, 2])
}

# ---------- NO → Variable key (more natural) ----------
node_key_grob <- function(var_names, blocks = 4, wrap_width = 16, fontsize = 8,
                          title = "Node number (NO) → variable") {
```

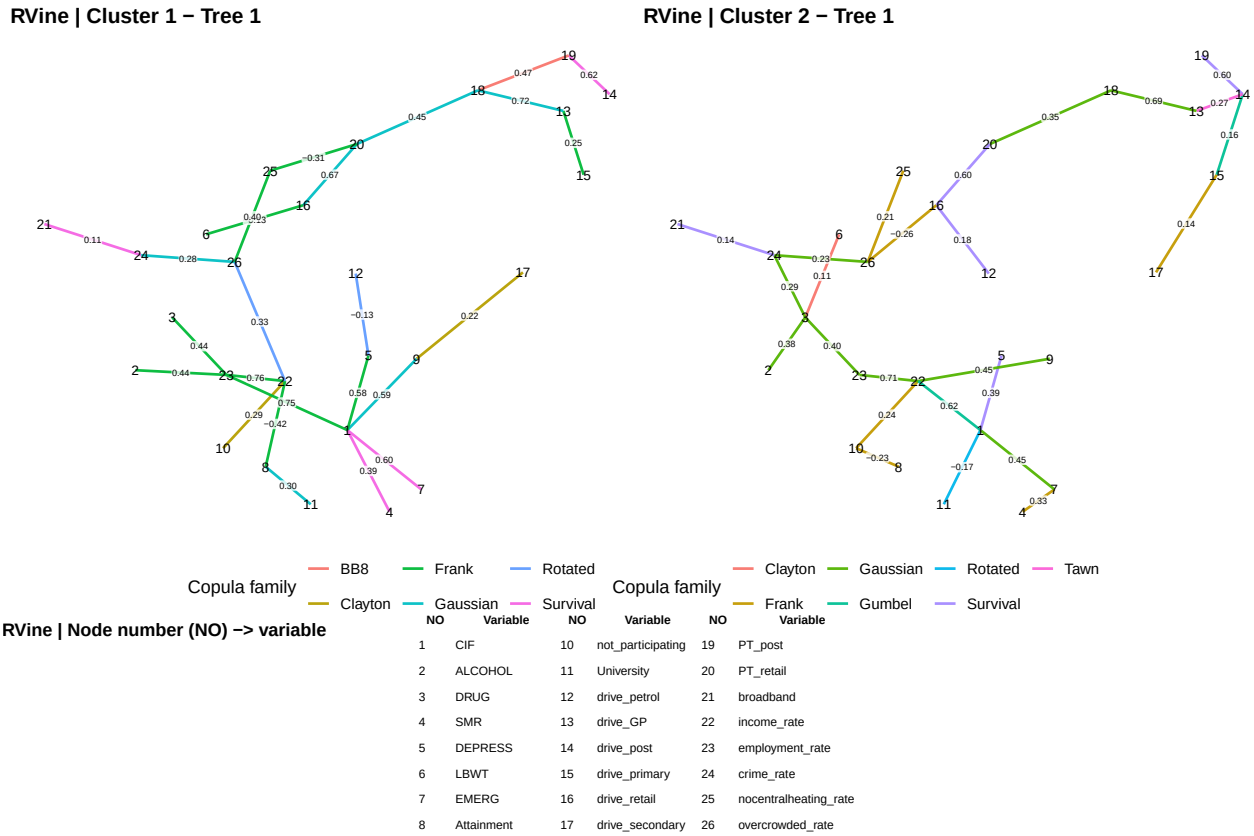
```r
  p <- length(var_names)
  nrow <- ceiling(p / blocks)

  NO  <- as.character(seq_len(p))
  Var <- str_wrap(as.character(var_names), width = wrap_width)

  cols <- vector("list", length = 2 * blocks)
  coln <- character(2 * blocks)

  for (b in seq_len(blocks)) {
    i_start <- (b - 1) * nrow + 1
    i_end   <- min(b * nrow, p)
    take    <- i_start:i_end

    no_col  <- rep("", nrow)
    var_col <- rep("", nrow)

    if (length(take) > 0) {
      no_col[seq_along(take)]  <- NO[take]
      var_col[seq_along(take)] <- Var[take]
    }

    cols[[2*b - 1]] <- no_col
    cols[[2*b]]     <- var_col
    coln[2*b - 1] <- "NO"
    coln[2*b]     <- "Variable"
  }

  df <- as.data.frame(cols, stringsAsFactors = FALSE)
  names(df) <- coln

  tg <- gridExtra::tableGrob(
    df, rows = NULL,
    theme = gridExtra::ttheme_minimal(
      base_size = fontsize,
      core = list(
        fg_params = list(hjust = 0, x = 0.02),
        bg_params = list(fill = NA, col = NA)
      ),
      colhead = list(
        fg_params = list(fontface = "bold"),
        bg_params = list(fill = NA, col = NA)
      )
    )
  )

  gridExtra::arrangeGrob(
    grid::textGrob(title, x = 0, just = "left", gp = grid::gpar(fontface = "bold")),
    tg,
    heights = grid::unit.c(grid::unit(1.2, "lines"), grid::unit(1, "null"))
  )
}
```

```r
# ---------- Tree plot: Tree>=2 default edge label is conditional NO ----------
plot_tree_ggraph <- function(RVM, tree = 1, title = "",
                             coords = NULL,
                             label_min_abs_tau = 0.0,
                             digits = 2,
                             label_zero_tau = FALSE,
                             edge_label = c("auto", "tau", "cond_no", "tau+cond_no", "none")) {

  edge_label <- match.arg(edge_label)
  if (edge_label == "auto") edge_label <- if (tree == 1) "tau" else "cond_no"

  vn <- RVM$names
  p  <- length(vn)
  nodes <- tibble(name = as.character(seq_len(p)), var = vn)

  ed <- extract_tree_edges(RVM, tree = tree, var_names = vn) %>%
    mutate(
      from = as.character(idx1),
      to   = as.character(idx2),
      lab_tau  = formatC(tau, format = "f", digits = digits),
      lab_cond = vapply(cond_idx, format_cond_no, character(1)),
      is_indep = abs(tau) < 1e-12,

      # ---- NEW LOGIC ----
      # If you're showing conditioning (cond_no or tau+cond_no), then
      # always show conditioning labels even for independence (dashed) edges.
      # Only apply abs-tau threshold to tau labels, not to conditioning labels.
      lab = case_when(
        edge_label == "none" ~ "",

        edge_label == "tau" ~ case_when(
          abs_tau < label_min_abs_tau ~ "",
          (!label_zero_tau & is_indep) ~ "",
          TRUE ~ lab_tau
        ),

        edge_label == "cond_no" ~ lab_cond,  # show for ALL edges, including dashed

        edge_label == "tau+cond_no" ~ case_when(
          # tau part only if strong enough (and not suppressed zeros)
          abs_tau >= label_min_abs_tau & (label_zero_tau | !is_indep) ~
            paste0(lab_tau, ifelse(lab_cond == "", "", paste0(" ", lab_cond))),
          # otherwise still show conditioning (even if tau small/zero)
          TRUE ~ lab_cond
        ),

        TRUE ~ ""
      )
    )

  g <- igraph::graph_from_data_frame(
    d = ed %>% select(from, to, family, tau, abs_tau, lab),
    directed = FALSE,
```

```r
    vertices = nodes
  )

  if (is.null(coords)) {
    set.seed(1)
    xy <- igraph::layout_with_fr(g)
    coords <- tibble(name = igraph::V(g)$name, x = xy[, 1], y = xy[, 2])
  }

  ord <- match(igraph::V(g)$name, coords$name)
  x_v <- coords$x[ord]
  y_v <- coords$y[ord]

  edf <- igraph::as_data_frame(g, what = "edges") %>%
    mutate(
      x    = coords$x[match(from, coords$name)],
      y    = coords$y[match(from, coords$name)],
      xend = coords$x[match(to,   coords$name)],
      yend = coords$y[match(to,   coords$name)],
      xm   = (x + xend) / 2,
      ym   = (y + yend) / 2
    )

  ggraph(g, layout = "manual", x = x_v, y = y_v) +
    geom_segment(
      data = edf %>% filter(abs(tau) < 1e-12),
      aes(x = x, y = y, xend = xend, yend = yend),
      colour = "grey80", linetype = "dashed", linewidth = 0.6, alpha = 0.9,
      inherit.aes = FALSE
    ) +
    geom_segment(
      data = edf %>% filter(abs(tau) >= 1e-12),
      aes(x = x, y = y, xend = xend, yend = yend, colour = family),
      linewidth = 0.85, alpha = 0.95,
      inherit.aes = FALSE
    ) +
    geom_label(
      data = edf %>% filter(lab != ""),
      aes(x = xm, y = ym, label = lab),
      size = 2.2, label.size = 0, fill = "white", alpha = 0.80,
      label.padding = unit(0.05, "lines"),
      inherit.aes = FALSE
    ) +
    geom_node_text(aes(label = name), size = 3) +
    scale_x_continuous(expand = expansion(mult = 0.06)) +
    scale_y_continuous(expand = expansion(mult = 0.06)) +
    coord_cartesian(clip = "on") +
    theme_void(base_size = 12) +
    ggtitle(title) +
    guides(colour = guide_legend(title = "Copula family")) +
    theme(plot.title = element_text(face = "bold"),
          legend.position = "bottom")
}
```

```r
# ---------- Side-by-side wrapper (with node key underneath) ----------
plot_tree_side_by_side <- function(fits, tree = 1, main_prefix = "",
                                   label_min_abs_tau = 0.0, digits = 2,
                                   show_node_key = TRUE,
                                   key_blocks = 3, key_wrap_width = 16, key_fontsize = 8,
                                   edge_label = "auto") {

  R1 <- fits[["cluster_1"]]$RVM
  R2 <- fits[["cluster_2"]]$RVM
  coords <- make_reference_layout(R1, R2, tree = tree, seed = 1)

  # sensible default thresholds: Tree 1 higher, Tree >= 2 lower
  if (is.null(label_min_abs_tau)) label_min_abs_tau <- if (tree == 1) 0.20 else 0.10

  p1 <- plot_tree_ggraph(R1, tree = tree,
                         title = paste0(main_prefix, "Cluster 1 - Tree ", tree),
                         coords = coords,
                         label_min_abs_tau = label_min_abs_tau,
                         digits = digits,
                         edge_label = edge_label)

  p2 <- plot_tree_ggraph(R2, tree = tree,
                         title = paste0(main_prefix, "Cluster 2 - Tree ", tree),
                         coords = coords,
                         label_min_abs_tau = label_min_abs_tau,
                         digits = digits,
                         edge_label = edge_label)

  top <- (p1 | p2) + plot_layout(guides = "collect") &
    theme(legend.position = "bottom")

  if (!show_node_key) return(top)

  key <- wrap_elements(
    full = node_key_grob(R1$names, blocks = key_blocks,
                         wrap_width = key_wrap_width, fontsize = key_fontsize,
                         title = paste0(main_prefix, "Node number (NO) → variable"))
  )

  (top / key) + plot_layout(heights = c(1, 0.38))
}

# =====================
# Example usage:
# =====================
# Tree 1 (tau labels):
plot_tree_side_by_side(fits_rvine, tree = 1, main_prefix = "RVine | ")
```

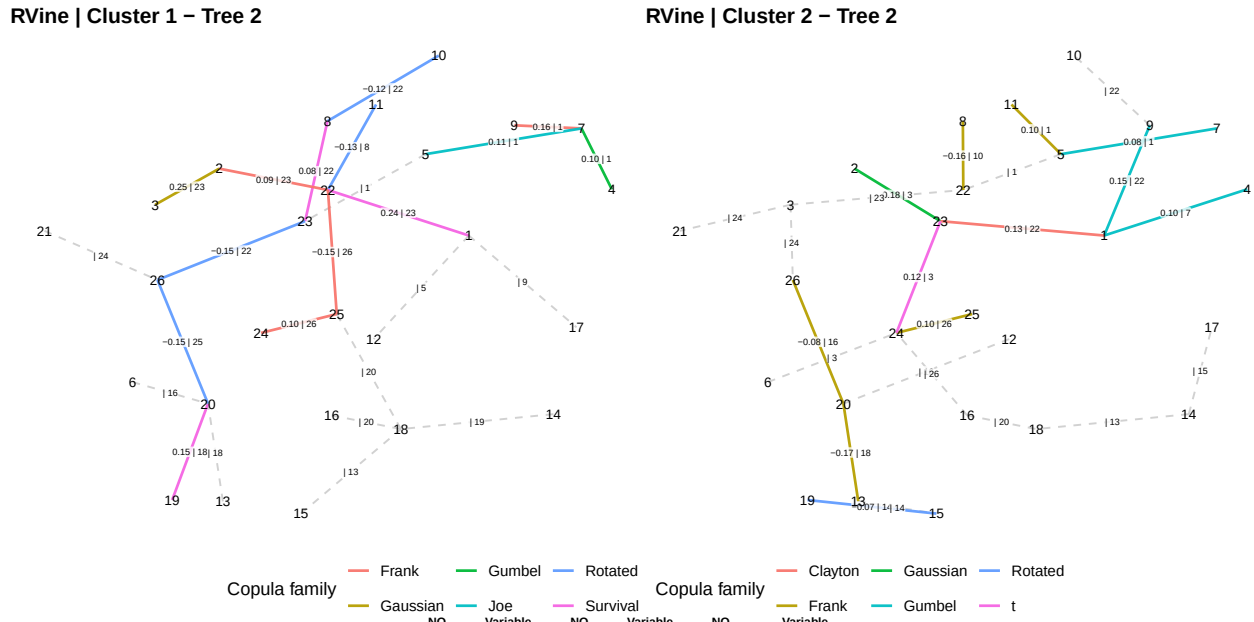
```
## Warning: The `label.size` argument of `geom_label()` is deprecated as of ggplot2 3.5.0.
## i Please use the `linewidth` argument instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



RVine | Node number (NO) –> variable

NO	Variable	NO	Variable	NO	Variable
1	CIF	10	not_participating	19	PT_post
2	ALCOHOL	11	University	20	PT_retail
3	DRUG	12	drive_petrol	21	broadband
4	SMR	13	drive_GP	22	income_rate
5	DEPRESS	14	drive_post	23	employment_rate
6	LBWT	15	drive_primary	24	crime_rate
7	EMERG	16	drive_retail	25	nocentralheating_rate
8	Attainment	17	drive_secondary	26	overcrowded_rate

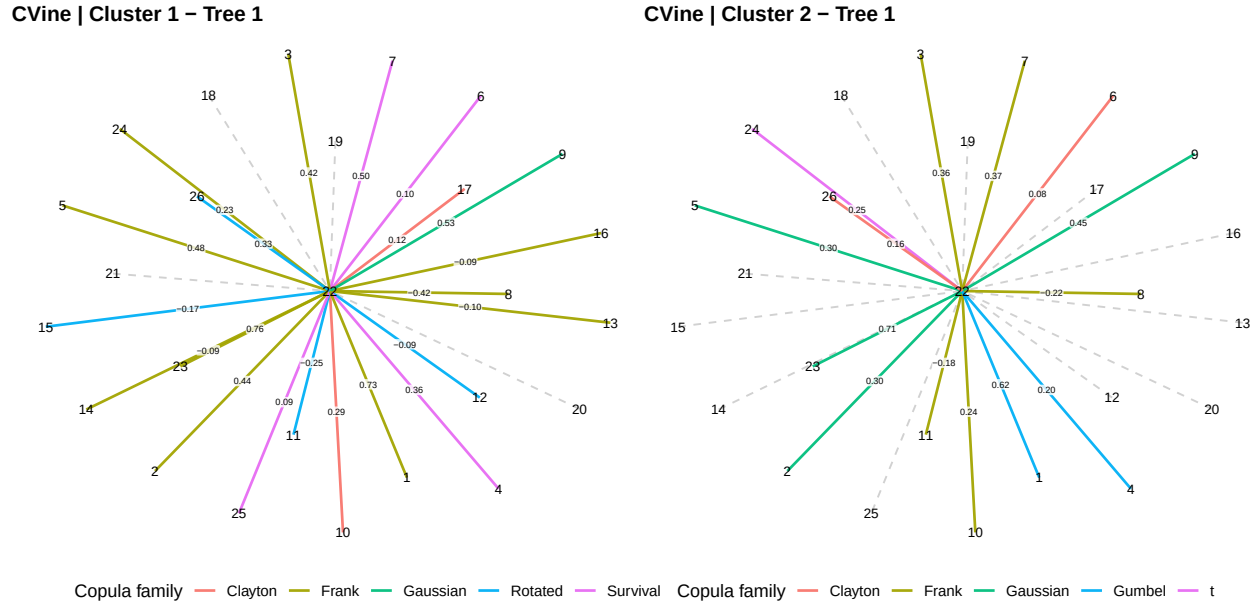
```
plot_tree_side_by_side(fits_rvine, tree = 2, main_prefix = "RVine | ",
                       edge_label = "tau+cond_no")
```



RVine | Node number (NO) –> variable

NO	Variable	NO	Variable	NO	Variable
1	CIF	10	not_participating	19	PT_post
2	ALCOHOL	11	University	20	PT_retail
3	DRUG	12	drive_petrol	21	broadband
4	SMR	13	drive_GP	22	income_rate
5	DEPRESS	14	drive_post	23	employment_rate
6	LBWT	15	drive_primary	24	crime_rate
7	EMERG	16	drive_retail	25	nocentralheating_rate
8	Attainment	17	drive_secondary	26	overcrowded_rate

```
plot_tree_side_by_side(fits_cvine, tree = 1, main_prefix = "CVine | ")
```

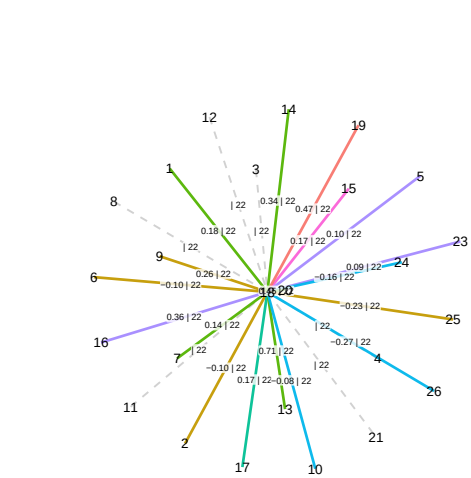


CVine | Node number (NO) –> variable

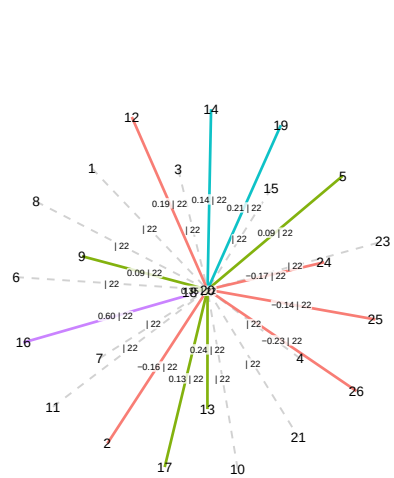
NO	Variable	NO	Variable	NO	Variable
1	CIF	10	not_participating	19	PT_post
2	ALCOHOL	11	University	20	PT_retail
3	DRUG	12	drive_petrol	21	broadband
4	SMR	13	drive_GP	22	income_rate
5	DEPRESS	14	drive_post	23	employment_rate
6	LBWT	15	drive_primary	24	crime_rate
7	EMERG	16	drive_retail	25	nocentralheating_rate
8	Attainment	17	drive_secondary	26	overcrowded_rate

```r
plot_tree_side_by_side(fits_cvine, tree = 2, main_prefix = "CVine | ",
                       edge_label = "tau+cond_no")
```

CVine | Cluster 1 − Tree 2



CVine | Cluster 2 − Tree 2



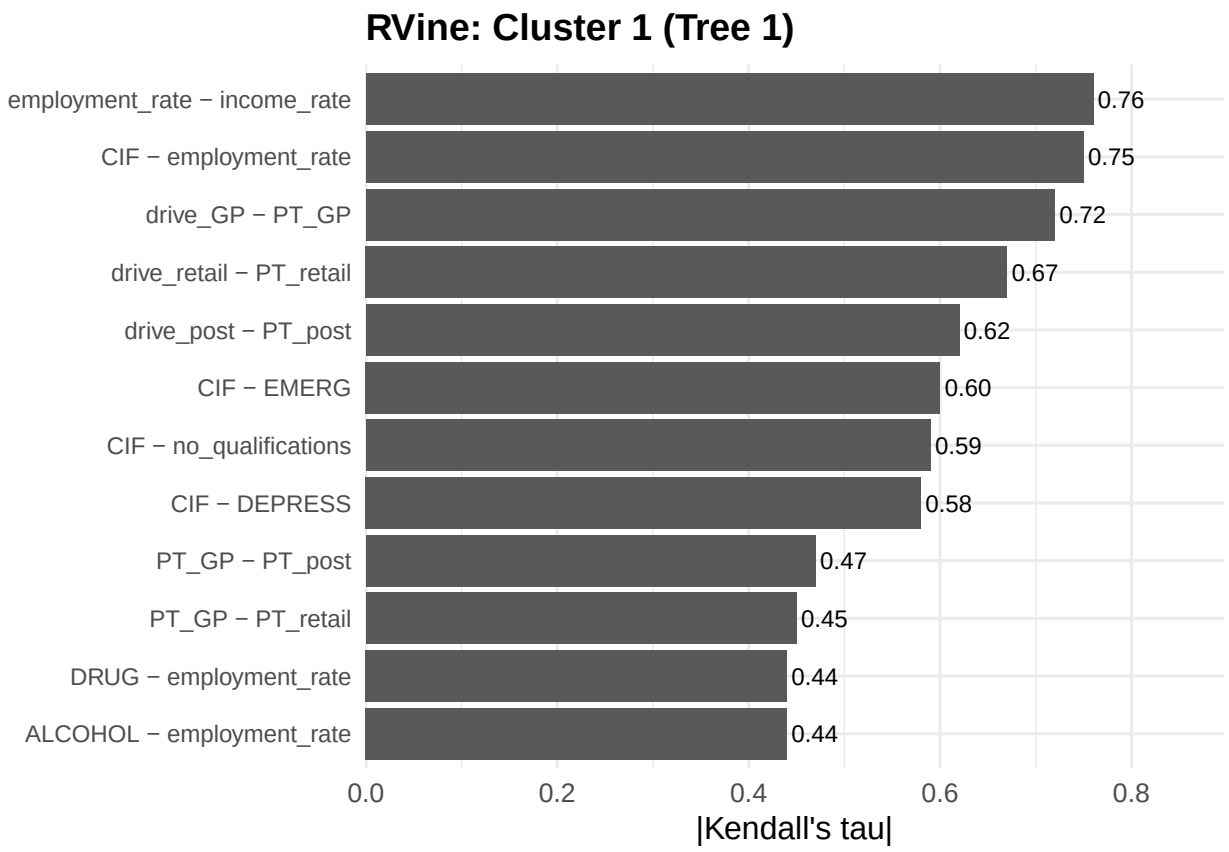
CVine | Node number (NO) –> variable

NO	Variable	NO	Variable	NO	Variable
1	CIF	10	not_participating	19	PT_post
2	ALCOHOL	11	University	20	PT_retail
3	DRUG	12	drive_petrol	21	broadband
4	SMR	13	drive_GP	22	income_rate
5	DEPRESS	14	drive_post	23	employment_rate
6	LBWT	15	drive_primary	24	crime_rate
7	EMERG	16	drive_retail	25	nocentralheating_rate
8	Attainment	17	drive_secondary	26	overcrowded_rate

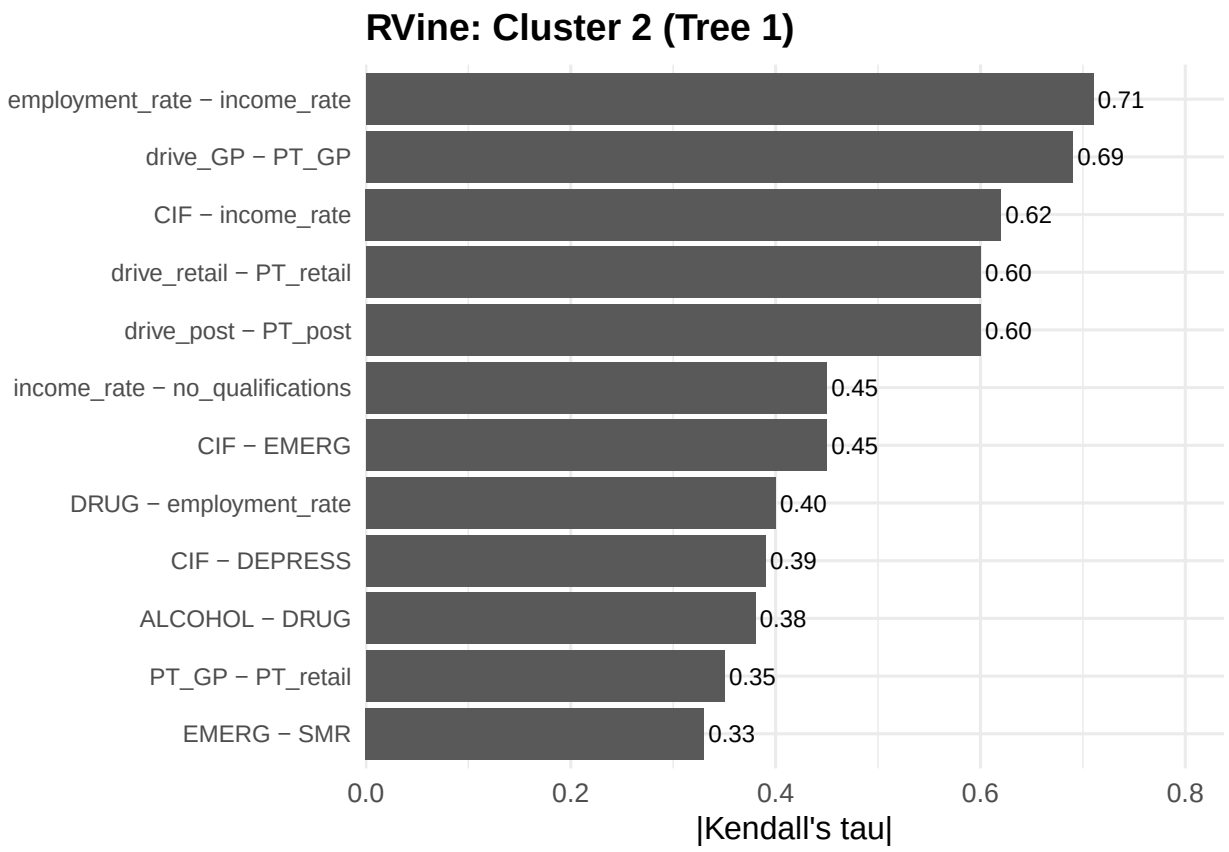
```r
# ========================================
# Accurate bar charts: Cluster 1, Cluster 2, and Difference
# (robust to cluster naming and A-B vs B-A duplicates)
# ========================================

library(dplyr)
library(tidyr)
library(stringr)
library(ggplot2)

# Canonicalise pair key so A-B and B-A are identical
canon_pair <- function(a, b, sep = " - ") {
  ifelse(a <= b, paste(a, b, sep = sep), paste(b, a, sep = sep))
}

# Normalise cluster labels to C1/C2 (robust)
norm_cluster <- function(x) {
  x0 <- tolower(trimws(as.character(x)))
  dplyr::case_when(
    x0 %in% c("cluster_1", "cluster 1", "c1", "1") ~ "C1",
    x0 %in% c("cluster_2", "cluster 2", "c2", "2") ~ "C2",
    TRUE ~ x  # fallback (shouldn't happen)
  )
}
```

```r
# Build clean edge table for a given tree
build_tree_edges <- function(fits, tree = 1, wrap_width = 42) {
  purrr::imap_dfr(fits, function(obj, nm) {
    extract_tree_edges(obj$RVM, tree = tree, var_names = obj$RVM$names) %>%
      mutate(cluster_raw = nm)
  }) %>%
    mutate(
      cluster = norm_cluster(cluster_raw),
      pair_key = canon_pair(var1, var2),
      pair_lab = str_wrap(pair_key, width = wrap_width)
    ) %>%
    select(cluster, pair_key, pair_lab, tau, abs_tau) %>%
    distinct()
}

# Plot top-n for ONE cluster (separate chart)
plot_top_abs_tau_cluster <- function(edges, cluster = "C1",
                                     n = 12, title = NULL, digits = 2) {
  lab <- ifelse(cluster == "C1", "Cluster 1", "Cluster 2")

  df <- edges %>%
    filter(cluster == !!cluster) %>%
    # in case the same pair appears twice, keep the max |tau|
    group_by(pair_key, pair_lab) %>%
    summarise(
      tau = tau[which.max(abs_tau)],
      abs_tau = max(abs_tau),
      .groups = "drop"
    ) %>%
    slice_max(abs_tau, n = n, with_ties = FALSE) %>%
    arrange(abs_tau) %>%
    mutate(pair_lab = factor(pair_lab, levels = pair_lab))

  ggplot(df, aes(x = abs_tau, y = pair_lab)) +
    geom_col() +
    geom_text(aes(label = formatC(tau, format = "f", digits = digits)),
              hjust = -0.10, size = 3) +
    scale_x_continuous(expand = expansion(mult = c(0, 0.18))) +
    labs(
      x = "|Kendall's tau|",
      y = NULL,
      title = title %||% paste0(lab, ": Top ", n, " edges by |tau|")
    ) +
    theme_minimal(base_size = 12) +
    theme(plot.title = element_text(face = "bold"),
          axis.text.y = element_text(size = 9))
}

# Plot difference chart: Δ|tau| = |tau|_C2 - |tau|_C1
plot_top_abs_tau_absdiff <- function(edges, n = 12, title = NULL, digits = 2, wrap_width = 34) {

  wide <- edges %>%
    dplyr::select(cluster, pair_key, pair_lab, abs_tau) %>%
```

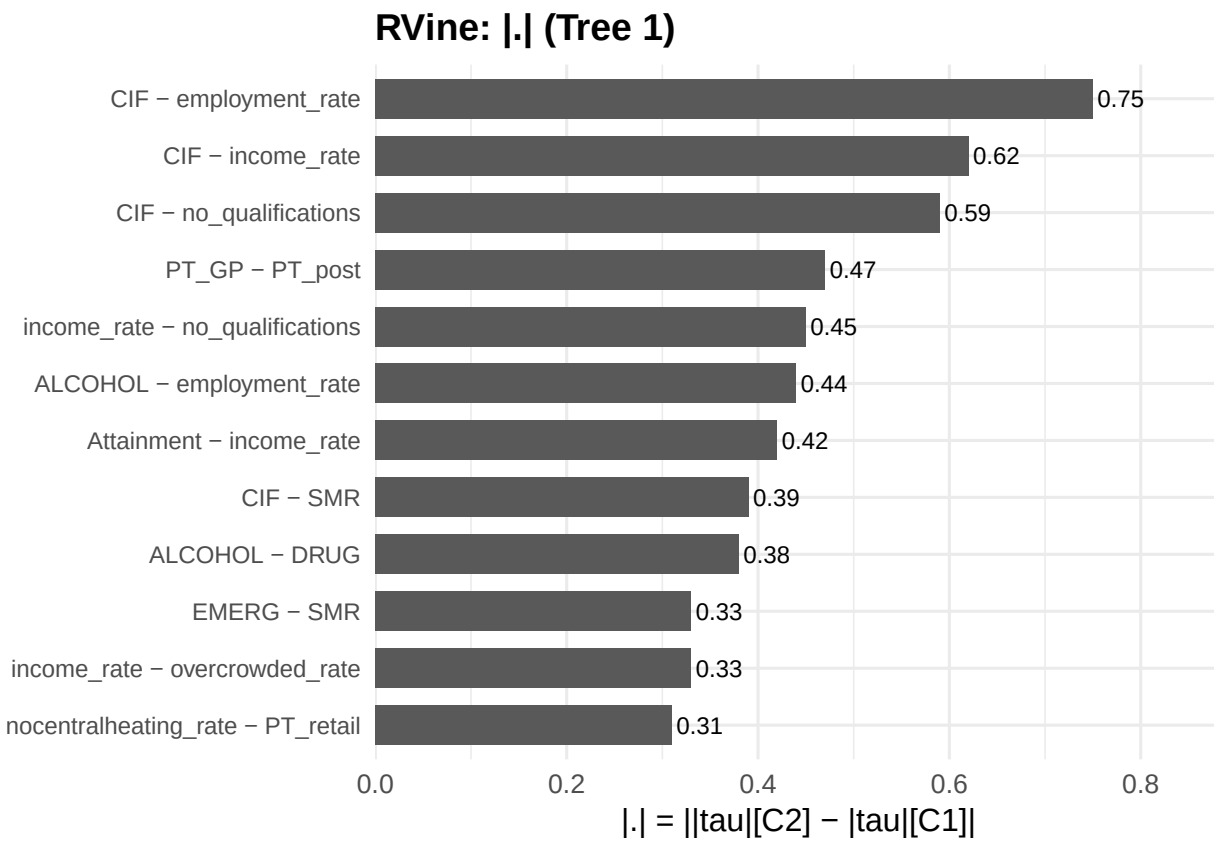
```r
    dplyr::distinct() %>%
    tidyr::pivot_wider(
      names_from  = cluster,
      values_from = abs_tau,
      values_fill = 0,
      names_prefix = "abs_tau_"
    )

  # safety: if any column is absent, create it as 0
  if (!"abs_tau_C1" %in% names(wide)) wide$abs_tau_C1 <- 0
  if (!"abs_tau_C2" %in% names(wide)) wide$abs_tau_C2 <- 0

  wide <- wide %>%
    dplyr::mutate(
      absdiff  = abs(abs_tau_C2 - abs_tau_C1),
      pair_lab = stringr::str_wrap(pair_lab, width = wrap_width),
      lab      = formatC(absdiff, format = "f", digits = digits)
    ) %>%
    dplyr::slice_max(absdiff, n = n, with_ties = FALSE) %>%
    dplyr::arrange(absdiff) %>%
    dplyr::mutate(pair_lab = factor(pair_lab, levels = pair_lab))

  ggplot2::ggplot(wide, ggplot2::aes(x = absdiff, y = pair_lab)) +
    ggplot2::geom_col(width = 0.70) +
    ggplot2::geom_text(ggplot2::aes(label = lab), hjust = -0.10, size = 3) +
    ggplot2::scale_x_continuous(expand = ggplot2::expansion(mult = c(0, 0.18))) +
    ggplot2::labs(
      x = expression("|Δ| = ||tau|[C2] - |tau|[C1]|"),
      y = NULL,
      title = title %||% paste0("Top ", n, " edges by |Δ| in |tau|")
    ) +
    ggplot2::theme_minimal(base_size = 12) +
    ggplot2::theme(
      plot.title = ggplot2::element_text(face = "bold"),
      axis.text.y = ggplot2::element_text(size = 9)
    )
}




# -------------------------
# Example usage:
tree_edges <- build_tree_edges(fits_rvine, tree = 1)

p_c1 <- plot_top_abs_tau_cluster(tree_edges, cluster = "C1", n = 12, title = "RVine: Cluster 1 (Tree 1)
p_c2 <- plot_top_abs_tau_cluster(tree_edges, cluster = "C2", n = 12, title = "RVine: Cluster 2 (Tree 1)
p_absdiff <- plot_top_abs_tau_absdiff(tree_edges, n = 12, title = "RVine: |Δ| (Tree 1)")

p_c1
```


RVine: Cluster 1 (Tree 1)
employment_rate − income_rate
CIF − employment_rate
drive_GP − PT_GP
drive_retail − PT_retail
drive_post − PT_post
CIF − EMERG
CIF − no_qualifications
CIF − DEPRESS
PT_GP − PT_post
PT_GP − PT_retail
DRUG − employment_rate
ALCOHOL − employment_rate
0.76
0.75
0.72
0.67
0.62
0.60
0.59
0.58
0.47
0.45
0.44
0.44
0.0
0.2
0.4
0.6
0.8
|Kendall's tau|

```
p_c2
```


RVine: Cluster 2 (Tree 1)
employment_rate − income_rate
drive_GP − PT_GP
CIF − income_rate
drive_retail − PT_retail
drive_post − PT_post
income_rate − no_qualifications
CIF − EMERG
DRUG − employment_rate
CIF − DEPRESS
ALCOHOL − DRUG
PT_GP − PT_retail
EMERG − SMR
0.71
0.69
0.62
0.60
0.60
0.45
0.45
0.40
0.39
0.38
0.35
0.33
0.0
0.2
0.4
0.6
0.8
|Kendall's tau|

```
p_absdiff
```




```r
# ========================================
# CVine vs RVine comparison by cluster using logLik / AIC / BIC
# (fixes the NA issue by computing npar explicitly)
# Requires: fits_rvine, fits_cvine created earlier (each element has $U and $RVM)
# ========================================

library(dplyr)
library(purrr)
library(tibble)
library(VineCopula)
library(knitr)

# Count free parameters in a VineCopula RVineMatrix
# - 1 parameter for each non-independence pair-copula (family != 0)
# - +1 extra parameter whenever par2 is used (e.g., t, BB families), i.e., par2 != 0
count_vine_par <- function(RVM) {
  stopifnot(inherits(RVM, "RVineMatrix"))
  m <- lower.tri(RVM$family)
  fam  <- RVM$family[m]
  par2 <- RVM$par2[m]
  fam[is.na(fam)]   <- 0
  par2[is.na(par2)] <- 0
  sum(fam != 0) + sum(fam != 0 & abs(par2) > 1e-12)
}

# Robustly extract logLik from RVineLogLik()
get_rvine_loglik <- function(U, RVM) {
```

```r
  out <- VineCopula::RVineLogLik(U, RVM)
  if (!is.null(out$loglik)) return(as.numeric(out$loglik))
  if (!is.null(out$logLik)) return(as.numeric(out$logLik))
  if (!is.null(out$value))  return(as.numeric(out$value))
  if (is.numeric(out) && length(out) >= 1) return(as.numeric(out[1]))
  stop("Could not extract log-likelihood from RVineLogLik().")
}

# One row (cluster, model) -> nobs, npar, logLik, AIC, BIC
ic_one <- function(fit, model_name) {
  U <- fit$U
  RVM <- fit$RVM
  n <- nrow(U)
  k <- count_vine_par(RVM)
  ll <- get_rvine_loglik(U, RVM)
  tibble(
    model = model_name,
    nobs  = n,
    npar  = k,
    logLik = ll,
    AIC = -2 * ll + 2 * k,
    BIC = -2 * ll + log(n) * k
  )
}

# Build table (same format as your screenshot, but with AIC/BIC filled)
ic_tbl <- bind_rows(
  imap_dfr(fits_cvine, ~ ic_one(.x, "CVine") %>% mutate(cluster = .y)),
  imap_dfr(fits_rvine, ~ ic_one(.x, "RVine") %>% mutate(cluster = .y))
) %>%
  mutate(
    cluster = gsub("^cluster_", "Cluster ", cluster),
    logLik = round(logLik, 2),
    AIC    = round(AIC, 2),
    BIC    = round(BIC, 2)
  ) %>%
  select(cluster, model, nobs, npar, logLik, AIC, BIC) %>%
  arrange(cluster, model)

kable(ic_tbl, caption = "CVine vs RVine comparison by cluster (logLik/AIC/BIC)")
```

```
## Warning in attr(x, "align"): 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")
```

```
## Warning in attr(x, "format"): 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")
```

Table 1: CVine vs RVine comparison by cluster (logLik/AIC/BIC)

cluster	model	nobs	npar	logLik	AIC	BIC
Cluster 1	CVine	326	109	2999.56	-5781.11	-5368.34
Cluster 1	RVine	326	83	3026.16	-5886.33	-5572.01

cluster	model	nobs	npar	logLik	AIC	BIC
Cluster 2	CVine	370	95	2447.23	-4704.46	-4332.68
Cluster 2	RVine	370	84	2465.31	-4762.61	-4433.88

```r
# Optional: a compact "difference" table (negative means CVine is better = smaller)
ic_diff <- ic_tbl %>%
  tidyr::pivot_wider(names_from = model, values_from = c(logLik, AIC, BIC, npar)) %>%
  mutate(
    d_logLik = logLik_CVine - logLik_RVine,
    d_AIC    = AIC_CVine    - AIC_RVine,
    d_BIC    = BIC_CVine    - BIC_RVine
  ) %>%
  select(cluster, d_logLik, d_AIC, d_BIC)

kable(ic_diff, caption = "CVine - RVine differences (negative AIC/BIC => CVine preferred)")
```

```
## Warning in attr(x, "align"): 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")
## Warning in attr(x, "align"): 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")
```

Table 2: CVine − RVine differences (negative AIC/BIC => CVine preferred)

cluster	d_logLik	d_AIC	d_BIC
Cluster 1	-26.60	105.22	203.67
Cluster 2	-18.08	58.15	101.20